

Web QA Agent — Scope, Limits, and Blockers

What the agent does

Given a webpage URL, the agent:

- 1. **Maps** the page — captures every form field, dropdown, button, and label.
- 2. **Plans** test cases per field using domain-aware reasoning (e.g. apostrophes for names, plus-tagging for emails, length probes for free text).
- 3. **Executes** the tests, observes the app's response, and reports PASS/FAIL/BLOCKED with the actual error messages captured.

Output: structured JSON results + human-readable test report per run.

What works today

Apps that follow standard web accessibility patterns. Roughly 70–80% of typical business software.

Concrete proven examples this week:

App	Result
TECU credit union loan application (page 1)	16/24 PASS — email validation captured, real signal
Salesforce signup form	27/28 PASS — required + format validation caught on a live Fortune 500 app
Dolibarr ERP	Works
Standard HTML forms (DemoQA, Typeform, Google Forms, etc.)	Works

What "works" means in practice:

- Multiple test variations per field (empty, valid, edge cases, special chars)
 - Captures real validation error messages from the app
 - Restores form state between tests
 - Can navigate single-page forms autonomously
 - Generates a client-shareable test report
-

What doesn't work, and why

- 1. Apps using custom `<div>`-based form widgets

Examples: OrangeHRM (Oxd / Vue), some in-house admin panels.

The problem: these apps render text fields and dropdowns as styled `<div>` elements with JavaScript click handlers, instead of using real `<input>` or `<select>`. They look identical to a user but are invisible to standard automation.

Why our agent fails: the browser-automation tool we use sends synthetic clicks that these custom widgets refuse to honor — they expect real human input events that we can't generate from our current architecture.

2. Modern SPAs that regenerate IDs on each page load

Examples: Salesforce Lightning, Odoo (partial).

The problem: the field's internal name (its `id` attribute) changes every time the page reloads. By the time we run a test, the ID we memorized during extract is stale.

Where it still trips us up: apps that *a/so* lack stable label associations or use Shadow DOM end up with no anchor at all for our agent to grab onto.

3. Custom modal-based file uploaders

Example: TECU's "Upload Document" flow — clicking the upload button opens a modal where the user selects document type, then picks a file, then waits for OCR validation.

The problem: standard HTML file inputs (`<input type="file">`) work autonomously. TECU's pattern hides the file input behind multiple modal steps that our agent doesn't currently navigate.

Workaround today: user uploads files manually in Chrome before pressing Enter to extract/test the rest of the page.

4. Auth-gated apps in the chat UI

Examples: Forgenite, Odoo demo, anything behind a login.

The problem: the chat interface launches a fresh browser per command and can't pause for manual login. The agent ends up on the login page, sees no form fields to test, and reports zero results.

Workaround today: terminal interface with `--wait` flag. User logs in once in the agent's browser; cookies persist across subsequent runs. Same agent, just CLI not chat.

5. Multi-page wizards (Save & Continue between sections)

Examples: TECU loan application (6 pages), most onboarding flows, KYC forms.

The problem: each page submits to the next; agent currently tests ONE page per command and can't autonomously fill page N to advance to page N+1.

Workaround today: `--loop` mode keeps Chrome alive while the user manually fills + clicks "Save & Continue" between sections. Agent tests each page in turn.

6. Shadow DOM and cross-origin iframes

Examples: some Web Components, third-party embedded forms.

The problem: browser security prevents JavaScript from reading content inside cross-origin frames or shadow roots. This is a fundamental browser-level boundary, not a coding gap.

7. Forms embedded inside iframes (any kind)

Example: Forgenite's overview page — the actual form is rendered inside an iframe, our scanner only inspects the top-level document and found 0 inputs there.

The problem: our DOM scanner queries the top-level page directly. Forms inside iframes — even same-origin ones — aren't discovered.

8. Lazy-rendered / virtualized content

Examples: large data tables, infinite-scroll dashboards, accordion-style forms where only the open section is in the DOM.

The problem: elements not currently visible aren't in the DOM yet. Our scanner sees only what's rendered when extract runs.

Workaround today: user scrolls through the page before pressing Enter to extract.

9. Cascade dropdowns without declared dependencies

Example: TECU's Employer / Sector / Employment Type — the child dropdowns are disabled until the parent is filled. Agent doesn't know which parent unlocks which child.

The problem: dependency relationships aren't visible in the DOM. The app's logic is in JavaScript that we don't introspect.

Workaround today: declare dependencies manually in the per-app `defaults.json` config file. Agent reads them and fills parent → child in the right order.

10. Session-bound demo URLs

Examples: `demo.odoo.com` redirects to `demoN.odoo.com/...` with temporary session cookies; the URL works for ~30 minutes then 404s.

The problem: chat interface launches a fresh browser per command with no cookies. The session URL points nowhere by the time we hit it.

Workaround today: terminal interface with `--wait` lets the user navigate through the demo chooser manually before tests run.

11. Extraction noise (date pickers, placeholder labels)

Examples:

- Date/time pickers using spinner buttons get captured as fake dropdowns named '17', '16', '09'.
- Forms without proper <label> tags — placeholder text like "e.g. Job Title, LinkedIn URL" ends up as the "field name."

The problem: our scanner falls back to placeholder text when no label is found. Date spinner widgets look like single-character dropdowns to the scanner.

Effect on results: clutters the report with weird-looking entries that don't represent real fields. Tests on these always fail or block.

App-side security features that block any automation

These aren't fixable on our side — they're intentional defenses:

- **CAPTCHA** — requires human solving.
- **2FA** — requires phone/authenticator each session.
- **Bot-detection scripts** — some apps detect headless Chrome and refuse to load. Rare on internal tools, common on consumer-facing sites.
- **Rate limiting** — we hit this on long runs occasionally.
- **Apps that intentionally obscure inputs** — banking PIN pads, virtual keyboards, drag-and-drop password entry. Designed to defeat any automation.
- **Real-money / production environments** — we explicitly refuse to run against URLs containing "prod"/"production" without a manual override flag.

Operational issues we hit this week (workflow, not security)

These aren't security blockers but real friction worth disclosing:

- **MCP Chrome session vs user's Chrome.** When a developer logs into an app in their normal browser, the agent's Chrome doesn't share that session. Cookies live in separate profiles. Easy to confuse.
- **Stale locator caches.** When the app updates its UI between extract and execute, locators captured earlier may no longer match.
- **OCR validation timing.** TECU's document upload triggers a 30–60 second OCR validation. Tests that touch fields before OCR completes get unreliable results. Manual wait between sections is currently required.

How to classify any candidate app — green / yellow / red

We can run a **5-minute diagnostic** against any URL and place it in one of three buckets:

Bucket	What it looks like	Engagement
 Green	Standard HTML, proper labels, real <input>/<select>, accessible ARIA roles, stable IDs	Full service, works out of the box

Bucket	What it looks like	Engagement
 Yellow	Modern framework (React/Vue/Angular) with proper inputs but quirks like dynamic IDs, custom dropdowns, lazy-rendered content	Partial coverage with manual workarounds
 Red	Custom div-based forms, heavy Shadow DOM, cross-origin iframes, real-time auth challenges	Currently out of scope

Most enterprise SaaS, CMS platforms, internal admin tools, and standard public forms fall in green or yellow.

What we proved this week

End-to-end runs against:

- Salesforce signup form (27/28 PASS, live public app)
- TECU loan application page 1-3 (real bugs surfaced — TECU doesn't validate certain required fields inline)
- Dolibarr ERP (works)
- Forgenite Edit Bot (19/30 PASS — partial, documented limits)
- OrangeHRM (out-of-scope confirmed — Oxd custom-div pattern)

The agent now consistently surfaces **real application bugs** (missing required-field validation, weak email format checks, etc.) — that's its primary value, not just running tests for the sake of running them.

Summary

Works reliably on apps that follow modern web accessibility standards (the majority of business software); doesn't work on a small set of patterns (custom-div components, modal-based uploaders, deep wizard automation, iframe-embedded forms, dynamic-ID SPAs without label anchors).